Security Self-Assessment

Permission tiers, known risks and adversarial test results

Finpass team
FinTechathon Shenzhen, Topic 3: Wealth Advisory Agent

Version 1.1 October 20, 2026

Project	Finpass, a personal AI financial advisor for Indonesian retail investors
Document	Deliverable 5: security self-assessment
Basis	Reading of the source code and configuration in the `ai`, `backend`, `frontend` and `gowa` directories, the existing test suites, and 76 adversarial test cases we wrote against the backend and the AI service (Section 4). No independent penetration test.
Evidence	Every claim carries a file reference of the form `path:line` so a reviewer can check it. Anything we could not check is marked "Not verified".

Contents

1 Scope and system overview **2**

2 Permission-tier implementation **3**
2.1 Tiers that exist in the code . . . 3
2.2 How each control is enforced . . . 4
2.3 What this means in practice . . . 6

3 Known risks and remediation status **6**
3.1 How we rated . . . 6

4 Adversarial testing of the running code **13**
4.1 Method . . . 13
4.2 Results at a glance . . . 13
4.3 Findings . . . 13
4.4 Where the defects came from . . . 14
4.5 How to reproduce . . . 14
4.6 Catalogue of edge cases . . . 16
4.7 What this testing does not show . . . 19
4.8 Recommendations . . . 19

5 Remediation roadmap **19**

6 Limitations of this self-assessment **20**

1 Scope and system overview

This report covers the security of Finpass as it exists in our repository on October 20, 2026. It describes who can do what in the system, how that is enforced, and what we still plan to improve.

Summary

Finpass was built so that rules, not a language model, make every financial decision, and so that each user's data stays separate from everyone else's. The main protections in place are these:

- Passwords are stored with bcrypt. Sign-in uses short-lived access tokens and rotating refresh tokens, and a refresh token that is reused after rotation ends every session of that user.
- Every feature route requires a signed-in user, and every query on user-owned data filters on the user id taken from the token, never from the request.
- Phone numbers, WhatsApp messages, chat text and titles, profile answers and assessment data are encrypted in the application with AES-256-GCM, with keys that can be rotated.
- The WhatsApp webhook is accepted only with a valid HMAC signature. The AI service accepts only callers with the internal key.
- Security-relevant events go to an append-only, hash-chained audit table, and the chain is verified at start-up and every day.
- A daily job deletes data past its retention period, and email is sent only over an encrypted connection in production.
- The profile explanation is checked for recommendation wording and for numbers that the rules did not produce. The chat checks that every citation points to a real source and always carries a fixed disclaimer that the code, not the model, adds.
- Users can export their data and delete their account, which removes everything stored about them.
- CI runs secret scanning and vulnerability scanning on all three repositories, and containers run as a non-root user.
- Personal identifiers (national id, email, phone) are masked before a question reaches the model, prompt-injection attempts are blocked before any model call, answers are stripped of images, HTML and script links, and request bodies are size-limited.

Reading our own code for this report, we found twenty-two weaknesses. We fixed six of them during the audit and reduced eleven more, each with a test. Section 3 lists all of them with their status, and says plainly which ones code alone cannot close. We then attacked our own services with 76 hostile-input test cases. They found 17 further defects, 16 of them fixed and one accepted with a reason (Section 4).

Finpass has four parts that we run ourselves, and three outside services in the data path. Table 1 lists them with the boundary each one sits behind.

Table 1: Components and trust boundaries

Component	Technology	Role and boundary
Frontend	Next.js	Only origin the browser talks to. Proxies `/api/v1/*` to the API (`frontend/next.config.ts:39`).
API	Go, chi router, Postgres	All business rules, authentication and storage. Public internet boundary.

Component	Technology	Role and boundary
AI service	Python, FastAPI, ChromaDB	Profile explanation and the advisory chat (retrieval, LLM call, guardrail). Reachable only with a static internal key (`ai/src/core/security.py`).
WhatsApp gateway	GoWA, vendored open-source project	Sends the daily briefing and relays replies to the API. Authenticated by basic auth outbound and an HMAC signature inbound.
OpenRouter	External	Chat model and embeddings. Receives personal financial facts (risk R-03).
Cloudflare R2	External	Holds the packaged RAG index that the AI service downloads at startup.
Google, SMTP provider	External	Sign-in with Google; delivery of one-time email codes.

A request from a signed-in user follows this path: browser, frontend rewrite, API (JWT check, per-user queries), and for chat or profile explanations, the AI service. The AI service builds a prompt from the user's facts and retrieved documents, calls OpenRouter, and returns text. It has no tools and cannot write to any store (`ai/src/infrastructure/openrouter.py`). Rules, not the model, decide the profile, the product mix and the briefing content.

What we looked at

We read the authentication code, every route registration, the SQL for each user-owned table, configuration loading, the AI service's security and prompt code, the Dockerfiles, the CI workflows and the database migrations. We ran the tests after our fixes: the full Go suite passed, including the database-backed tests, which ran against a real PostgreSQL instance and were not skipped, and the full Python suite passed (333 tests, one expected failure that documents a known limit). We also ran `govulncheck`, `pip-audit`, `bandit` and `npm audit`.

What we did not look at

We did not test the deployed system. We did not inspect Railway, R2, GoWA or the SMTP account settings, and we did not scan the Go or Python dependencies with a vulnerability tool. Section 6 lists these gaps in full.

2 Permission-tier implementation

2.1 Tiers that exist in the code

The code has no role system. There is no admin, no expert or curator, and no support role. The access token carries one claim that matters, the user id (`backend/internal/features/auth/infrastructure/crypto.go:36`), and no route checks anything else. What we call tiers below are the distinct kinds of caller the code can tell apart. We derived them from the route registrations and the authentication code, not from a design document.

Table 2: Caller tiers found in the code

Tier	Caller	How the code recognizes it
T0	Anonymous	No credential. Can reach the auth endpoints (limited per IP), `/health`, and the WhatsApp webhook (which then needs a valid HMAC signature). The demo notes routes are also open, but only outside production.
T1	Signed-in user	Bearer JWT, HS256, 15 minute lifetime. The user id comes only from the validated token (`backend/internal/features/auth/transport/session.go:20-61`).
T2	Internal service	Header `X-Internal-Api-Key`, one static shared secret, compared in constant time (`ai/src/core/security.py`). Used by the API to call the AI service.
T3	Operator	Anyone with Railway, database, R2 or key access. Not an application role, but it decides who can read what, so we include it.

Table 3 shows what each tier can do with each resource. "R" is read, "W" is write, "X" is call or execute, "own" means only rows whose `user_id` equals the caller's id, and a dash means no access through the application.

Table 3: Tier against resource

Resource	T0	T1	T2	T3
Register, login, reset, refresh	X	X	–	–
Own profile, answers, assessments	–	R/W own	–	R/W, plaintext
Own portfolio holdings	–	R/W own	–	R/W, plaintext
Own chat sessions and messages	–	R, delete own; text written by the system	–	Ciphertext; plaintext only with the key
Own WhatsApp settings and phone	–	R/W own	–	Phone and message bodies are ciphertext
Market data, news, discover	–	R	–	R/W
Notes (demo feature, not served in production)	–	–	–	R/W
AI endpoints `/ml/*`	–	–	X	X
WhatsApp webhook	X with valid HMAC	–	–	–
RAG corpus (index)	–	–	R	W (R2 access)
Audit trail	–	–	–	R/W (database)
Secrets and encryption keys	–	–	–	R/W

The notes row shows the production behavior. We found the demo notes routes open to anyone while reading the code and stopped serving them in production (risk R-05). Two rows are still open: several sensitive columns are plaintext to T3 (risk R-08), and the repository does not say who may write to the R2 bucket (risk R-13).

2.2 How each control is enforced

Table 4 lists the permission and protection controls, where each one lives, and our status for it. "Implemented" means the code does it and we read the code. "Partial" means it exists with a gap that we

describe in Section 3. “Not implemented” means we found nothing. We did not use “Planned” for any control, because we have not committed to a schedule.

Table 4: Controls, enforcement point and status

Control and enforcement point	Reference	Status
Who is the caller		
Authentication on every feature route: `RequireAuth` middleware applied with `r.Use(h.auth)` in each feature's router	`backend/internal/features/*/transport/handler.go`; `auth/transport/session.go:47`	**Implemented**; the demo notes routes are not served in production (`backend/internal/app/app.go:65`)
Password storage: bcrypt cost 12, length 8–72, one letter and one digit	`backend/internal/app/app.go:103`; `auth/domain/user.go:48-58`	**Implemented** (no breached-password check)
Sessions: 15 minute JWT plus rotating refresh token, only the SHA-256 stored; reuse after 10 seconds revokes all sessions of the user	`auth/domain/session.go:22-30`; `auth/application/session.go:10-33`	**Implemented**
Email codes: 6 digits, hashed, 10 minute life, 5 attempts, 60 second resend gap	`auth/domain/otp.go:21-23`	**Implemented**
Internal service authentication: static key in a header	`ai/src/core/security.py`	**Implemented** (one shared key, no rotation)
Webhook authentication: HMAC-SHA256 over the raw body, constant-time compare, empty secret rejects everything	`whatsapp/domain/phone.go:83-99`	**Implemented**
Multi-factor authentication	none found	**Not implemented**
What may the caller touch		
Row ownership: every query on a user-owned table filters on `user_id`, taken from the token	`backend/internal/features/*/infrastructure/*.go`	**Implemented** at application level
Database-level isolation (row-level security)	no policy in `backend/db/migrations`	**Not implemented**
Role-based tiers (admin, expert, support)	none in code	**Not implemented**
Chat session ownership check before reading messages	`advisory/application/service.go:183-187`	**Implemented**; the message query itself is not user-scoped (`advisory/infrastructure/postgres.go:113,121`)
Limits and hygiene		
Login lockout (5 failures per account per 15 minutes), mail limit, export limit, chat limit of 20 per minute	`auth/transport/handler.go:37-39`; `advisory/application/service.go:17`	**Partial**: in memory, per instance; the per-IP key honors a configured proxy count (`TRUSTED_PROXY_HOPS`)
Per-IP limit of 30 per minute on the auth group	`backend/internal/platform/httpx/ratelimit.go:83-97`; `backend/internal/platform/httpx/clientip.go`	**Partial**: correct behind a proxy once `TRUSTED_PROXY_HOPS` is set; it defaults to 0
Request validation: 1 MiB cap, unknown JSON fields rejected, struct tags	`backend/internal/platform/httpx/decode.go:15-33`	**Implemented**
Response headers (API: HSTS, nosniff, no framing, no caching; frontend: same plus CSP)	`backend/internal/platform/httpx/security.go:6-16`; `frontend/next.config.ts:7-30`	**Partial**: frontend CSP is Report-Only and allows inline scripts
Column encryption (AES-256-GCM): phone numbers, WhatsApp messages, chat text and titles, profile answers and assessments	`backend/internal/platform/crypto/field.go`; `whatsapp/infrastructure/postgres.go`; `advisory/infrastructure/postgres.go`	**Partial**: holdings, email and name are plaintext

Control and enforcement point	Reference	Status
Audit trail: append-only table, hash chain, checked at start-up and daily	`backend/db/migrations/20261022000002_create_audit_events.sql`; `platform/audit/audit.go`; `platform/maintenance/maintenance.go`	**Implemented**
Log redaction by field name	`backend/internal/platform/logger/logger.go:10-20`	**Partial**: free text is not scanned
Data export and account deletion	`auth/transport/handler.go:220-248`	**Partial**: export omits watchlist, bookmarks and assessment history
Production start-up checks: `APP_ENV` must be set, and production refuses default secrets, missing keys, missing SMTP and `sslmode=disable`	`backend/internal/platform/config/config.go:121-143`	**Implemented**; deployed values not verified
Retention: a daily purge of expired codes, sessions, chats, WhatsApp messages and events	`backend/internal/platform/maintenance/maintenance.go`	**Implemented**

2.3 What this means in practice

A normal signed-in user cannot read another user's data through the API, as far as we can tell from the code, and the tests that cover the routes pass. The id is never taken from the request body, and every query we read carries the user predicate. That protection lives in the application only. One query that forgets the predicate would leak data, and the database offers no second barrier. The chat message query is the one place where we noticed the scoping depends on an earlier check in a different layer.

The AI service trusts one shared secret. It has no notion of a user, so a caller with the key can send any user context. That is acceptable only while the service is reachable from the API and nothing else. We could not verify the network setup (see `open-questions.md`).

We have no admin or expert tier. Expert-curated sources, if they matter to the product, are curated offline when the index is built. At query time the code only filters on a document status of `approved` and an unexpired date (`ai/src/infrastructure/advisor_chat/retrieval.py`). Nothing in the repository says who may change that metadata or write to the bucket.

3 Known risks and remediation status

3.1 How we rated

Likelihood and impact are each High, Medium or Low. Severity is our combined judgment of the risk before the mitigation in the table, not a calculated score; we did not use CVSS because most items are design points, not flaws in a package. Residual risk is what remains after the mitigation. Likelihood assumes an outsider or a curious user with no inside knowledge.

Severity labels, from worst to least: **Critical** **High** **Medium** **Low**. We found nothing we would call Critical, but we did not test the running system.

We acted on most of what we found while writing this report. Six items are fixed (R-01, R-05, R-14, R-19, R-20, R-21), eleven are reduced but not closed, one is a risk we accept (R-16), and four remain open or unverified. Every fix has a test, and the database-backed tests ran against a real PostgreSQL instance. The "Planned fix" column holds the fix we propose for what remains. We have not scheduled those, and Section 5 says so. Status is Fixed, Partial (mitigated, with a stated gap), Accepted (we chose to live with it, with the reason), Open (nothing done yet) or Not verified (we could not check

the deployed state). Some items cannot be closed by code alone, for example replacing the WhatsApp gateway or setting a spend cap at a provider; the table says which.

Table 5: Known-risk list

ID	Risk	Component	Lik.	Imp.	Severity	Current mitigation	Residual risk	Planned fix (proposed)	Status
R-01	**Account pre-hijack** (fixed). Someone could register a victim's email with their own password. If the victim later signed in with Google, the account was linked and verified, and the attacker's password still worked. (`auth/application/register.go:29-36`)	Auth: register, Google link	M	H	High	Linking Google removes a password that was set while the email was unverified (`auth/application/login.go:73-85`). The test `TestGoogleLinkDropsPasswordSetBeforeEmailWasVerified` fails without the change and passes with it.	Low. Someone could still race a person who is mid-signup by registering the same address again before the code is entered.	Store the password only once the emailed code is entered.	Fixed
R-02	**The chat can give advice we do not intend.** The model is told not to recommend products, but a prompt is not a guarantee.	AI chat	M	H	High	The chat answer is checked for wording that tells the user to buy or sell and for promises of guaranteed or risk-free profit, on Unicode-normalised text, and a failing answer is retried once and then replaced by source excerpts (`ai/src/domain/advisor_chat/guardrail.py`). Adversarial tests cover 10 evasion phrasings and 5 plain-education sentences (cases AI-33, AI-34); 8 of the 10 evasions passed the earlier filter (F-12). The product mix comes from rules, and the disclaimer is added by code.	Medium. The check is a phrase list in two languages. A differently worded recommendation can pass.	Grow the phrase list from reviewed real answers. Review a sample of stored answers before launch.	Partial
R-03	**Personal financial data goes to an outside LLM provider.** Each chat sends exact income, savings, installments, holdings with values and the question to OpenRouter. (`backend/internal/app/advisory_wiring.go:114-140`)	AI chat, OpenRouter	H	H	High	Name, email and phone are never sent, and national id, email and phone numbers typed into a question, the history or the stated goal are masked before the model or the search sees them (cases AI-23, AI-24; before F-01 they were sent). The request asks for providers that do not store or train on prompts. The default no longer lets OpenRouter fall back to other providers (`ai/src/core/config.py`), and the progress-title call no longer receives the question (`ai/src/infrastructure/observability/narrator.py`).	Medium. The feature needs the facts to answer, so this risk cannot be removed by code. The setting can still be overridden by an environment value.	Pin providers in production, state the transfer in the privacy notice, and review the provider's terms.	Partial

continued on the next page

ID	Risk	Component	Lik.	Imp.	Severity	Current mitigation	Residual risk	Planned fix (proposed)	Status
R-04	**Prompt injection, direct and through retrieved text.** The user's message, scraped news and document text reach one prompt.	AI chat	M	M	**Medium**	Facts and sources sit inside `<fakta>` and `<sumber>` blocks, and the system prompt says the blocks are data, not instructions (`ai/src/features/advisor_chat/service.py`). Text is Unicode-normalised and any tag-like token, with spaces or attributes, is removed, so it cannot close the blocks (9 breakout variants in 4 fields, case AI-22; 5 of the 9 got through before F-08). Questions that try to rewrite the rules are blocked before any model call (AI-17). Answers that repeat the prompt structure or contain a key are rejected, and images, HTML and script links are stripped (AI-25, AI-29). The model has no tools and no write access, and its output goes only to the same user.	Medium to Low. A crafted news item can still steer the wording of an answer, and obfuscated phrasing such as leetspeak is not caught (AI-19, an expected failure).	Add an injection test set and drop instruction-like lines from news before indexing. A teammate also works on this item.	Partial
R-05	**Public demo notes endpoints** (fixed). Anyone could create notes and read any note by id, from an in-memory map with no cap. (`backend/internal/app/app.go:59,65`)	API: notes	H	M	**Medium**	The routes are not served when `APP_ENV=production` (`backend/internal/app/app.go:65`). They remain in development as a template.	Low.	Delete the demo slice when no longer needed as a template.	Fixed
R-06	**Rate limits are weak.** The per-IP limit used the socket address, so behind a reverse proxy all users shared one bucket. Limits live in memory, per instance.	API: httpx, auth, chat	M	M	**Medium**	`TRUSTED_PROXY_HOPS` tells the API how many proxies add to `X-Forwarded-For`, and the limiter then uses the entry our proxy added, not one the client can forge (`platform/httpx/clientip.go`, tested). `/auth/refresh` and `/auth/logout` are now inside the limited group. Per-account lockout, mail limit, export limit and chat limits remain.	Medium until the value is set. It defaults to 0 because a wrong value lets clients forge the header. Limits are still per instance.	Set `TRUSTED_PROXY_HOPS` after confirming the hop count on the host. Use a shared store if the API runs on more than one instance.	Partial
R-07	**WhatsApp abuse.** A user could request codes for many numbers and send unlimited test messages, which can get our sending number banned. (`whatsapp/application/service.go:191,306-323`)	WhatsApp	M	M	**Medium**	Each user is capped at 5 verification codes and 5 test messages per hour whatever number is saved (`whatsapp/transport/handler.go:30-35`). Test: `TestCodesAndTestsAreCapped PerUser`. Sign-in is required, 5 attempts per code, and one verified number per account.	Low to Medium. The cap is in memory, per instance.	Move the cap to a shared store when the API scales out.	Partial

continued on the next page

ID	Risk	Component	Lik.	Imp.	Severity	Current mitigation	Residual risk	Planned fix (proposed)	Status
R-08	**Sensitive columns were plaintext.** Profile answers and assessment data were not encrypted in the application.	Database	L	H	**Medium**	`profiles.answers` and the input, trace and output of new assessments are sealed with AES-256-GCM; existing answers are sealed at start-up (`profile/infrastructure/postgres.go`). The test reads the raw columns from PostgreSQL and checks they hold no readable amounts, and that the profile, the assessment and the dashboard still read correctly. Phone numbers, WhatsApp messages, chat text and titles were already sealed.	Medium to Low. Email, name and holdings are still plaintext. Assessments written before this change cannot be rewritten because a trigger keeps them append-only, so they stay readable as stored.	Seal holdings and decide about email and name (the email is used for sign-in lookup).	Partial
R-09	**Secrets in the wrong place, one shared key, no rotation.** `ai/.env.production` holds names of backend secrets the AI service never reads, and `ai/.env` holds a Cloudflare token the AI config does not use. We read variable names only.	Secrets, config	M	H	**Medium**	The files are git-ignored and absent from history. Production refuses default secrets. The AI service now accepts a previous internal key during a rotation (`ai/src/core/security.py`, tested), so the key can change without downtime.	Medium. Rotating what already sat in the wrong file is a manual step we have not confirmed.	Split env files per service and rotate whatever sat in the wrong file. Then drop the previous-key setting.	Partial
R-10	**Production settings can drift to development.** `APP_ENV` used to default to `development`, which switched off every start-up check. The local `backend/.env.production` has debug logging, a localhost CORS origin, a service URL without a scheme and no `sslmode`; the deployed values may differ.	Config	M	H	**Medium**	`APP_ENV` is required. Production refuses `sslmode=disable` and warns about localhost CORS origins (`platform/config/config.go:121-143`). The backend image sets `APP_ENV=production`. Tests: `TestAppEnvIsRequired`, `TestProductionRejectsDatabaseWithoutTLS`.	Medium until the deployed values are read back.	Compare the deployed environment with these checks. Turn the CORS warning into a failure.	Partial
R-11	**Unofficial WhatsApp gateway.** GoWA drives WhatsApp Web with a paired phone session. WhatsApp can ban the number, and the session credentials live on the gateway. The code is vendored and was not reviewed. (`gowa/`)	WhatsApp	M	M	**Medium**	Opt-in, STOP and RESUME, basic auth to the gateway, signed webhook, and the new send caps (R-07).	Medium to High beyond a demo. Code cannot remove this.	Move to the official Business API before real users and review the vendored code.	Open
R-12	**AI service reachable with one key.** If the service has a public address, a static header is the only barrier. `/health` is open.	AI service	L	H	**Medium**	Constant-time key check. Production refuses the default key. API docs are off in production. Key rotation is supported (R-09).	Not known.	Keep it on private networking only. Add a timestamped signature per request.	Not verified

continued on the next page

ID	Risk	Component	Lik.	Imp.	Severity	Current mitigation	Residual risk	Planned fix (proposed)	Status
R-13	**RAG corpus can be tampered with.** Whoever can write to the bucket can change the answers.	RAG index, R2	L	H	**Medium**	The AI loader checks a SHA-256 for each index file against the manifest after every fresh download and stops on a mismatch (`ai/src/infrastructure/advisor_chat/loader.py:12-24`), and a script writes those hashes (`ai/scripts/hash_manifest.py`). Version and embedding model are checked at every start.	Medium. We read this code and did not test it. If a manifest has no hashes the loader only prints a warning, and a local copy is not re-checked at start because ChromaDB rewrites its database file while running.	Fail instead of warn when hashes are missing. Restrict write access to the bucket.	Partial
R-14	**Data was kept forever** (fixed). No purge existed for chats, WhatsApp messages, events, expired codes or sessions.	Database	H	M	**Medium**	A daily job deletes expired codes and sessions, chats after 365 days, WhatsApp messages after 90 days and events after 180 days; the periods are settable and 0 keeps data (`platform/maintenance/maintenance.go`, `platform/config/config.go`). The test runs against PostgreSQL and checks old rows go, messages cascade, and fresh rows stay. The audit trail is excluded on purpose.	Low.	Confirm the periods with whoever owns the privacy notice.	Fixed
R-15	**LLM cost and account farming.** Registration is open. One chat turn costs an embedding, a rerank, one or two LLM calls and several title calls.	AI chat, auth	M	M	**Medium**	20 chat messages per user per minute and now 200 per user per day (`advisory/application/service.go`, tested), email verification before sign-in, and the proxy-aware per-IP limit (R-06).	Medium. Limits are in memory, per instance, and we have not set a spend cap at the provider.	Set a spend cap at the model provider. Share the counters if the API scales out.	Partial
R-16	**Access tokens cannot be revoked.** The API only parses the JWT. Logout and reset revoke refresh sessions, not issued access tokens, which live 15 minutes.	Auth	L	M	**Low**	Short lifetime, rotating refresh tokens, and a reset that revokes all sessions.	Low. We accept it: a per-request check would add a database read to every call for a small window.	Revisit if the lifetime grows.	Accepted
R-17	**Weak browser-side containment.** The frontend CSP is Report-Only and allows inline scripts. (`frontend/next.config.ts:7-30`)	Frontend	L	M	**Low**	The access token is kept in memory, the refresh cookie is HttpOnly, and we found no `dangerouslySetInnerHTML` in the source.	Low. We did not switch the header to enforcing because we could not check it in a browser here.	Check the report-only violations, then enforce, then move to nonces.	Partial

continued on the next page

ID	Risk	Component	Lik.	Imp.	Severity	Current mitigation	Residual risk	Planned fix (proposed)	Status
R-18	**No multi-factor authentication.** Email access is enough to take over an account through the reset flow.	Auth	L	M	**Low**	Lockout, hashed codes, short code life, sessions revoked on reset.	Low.	Add TOTP for password accounts.	Open
R-19	**Thin audit trail** (fixed). Only sign-in, export and deletion were recorded, and the chain was never verified.	Audit	M	L	**Low**	Registration, email verification, password reset, Google sign-in, logout, consent changes, WhatsApp settings and number verification are now recorded (a test registers and logs out through the real router and reads the rows back). The chain is verified at start-up and daily, and an altered row is logged as a security error (`platform/maintenance/maintenance.go`).	Low. A database owner can still drop the trigger, and the head hash is not stored elsewhere.	Store the head hash outside the database.	Fixed
R-20	**Security scans were unpinned and unproven** (fixed). CI fetched scanners with `@latest` or version tags.	CI	L	M	**Low**	Actions are pinned to commit hashes and scanners to versions in all three repositories. We ran `govulncheck` (no reachable vulnerabilities), `pip-audit` (see R-22), `bandit` (no findings at medium or above) and `npm audit` (none) ourselves.	Low. We did not run `trivy`, and we have no CI run history.	Keep the passing runs as evidence.	Fixed
R-21	**Email used opportunistic TLS** (fixed). The mail library sent in plain text if the server did not offer STARTTLS.	Auth mailer	L	L	**Low**	Production refuses to send unless the connection is encrypted (`auth/infrastructure/mailer.go`). The test runs a server that never offers STARTTLS and checks that nothing is sent.	Low.	None.	Fixed
R-22	**Advisories in two Python dependencies.** `pip-audit` reports five advisories for ChromaDB 1.5.9 and one for NLTK 3.10.3, none with a published fixed version.	AI service dependencies	L	M	**Low**	The ChromaDB advisories concern its server and authorization providers. We use the embedded client, not the server, and NLTK's path issue needs a caller-controlled model path, which we do not pass. We believe neither is reachable. We did not test that.	Low to Medium.	Re-run the scan when a fixed version appears and upgrade.	Open

4 Adversarial testing of the running code

After the code reading in Section 3 we attacked our own services. This section describes 76 hostile-input edge cases, 46 against the AI service and 30 against the backend, written as automated tests that run on every build. They are an addition to the code reading, not a replacement for an independent penetration test (Section 6).

4.1 Method

For each case we wrote the test for the *secure* behaviour first, ran it against the code as it stood, and fixed what failed. The "Before" column in Table 7 and Table 8 is the result of that first run, so it shows what an attacker would have found. The tests drive the real HTTP router (the backend against a real PostgreSQL database) and fake only what is outside our control: the language model, the retriever and the mail and WhatsApp providers. No network, API key or model download is needed.

The cases fall into seven groups: access control and forged credentials, injection (SQL, prompt, markup), input validation and payload abuse, personal-data leakage, output safety, rate limiting and brute force, and availability. The backend tests also check every response of every test for leaks: no response may contain a database error, a stack trace, a file path or a secret.

4.2 Results at a glance

- **76 edge cases.** 50 held on the first run without any change, 24 exposed a defect, 1 is a known limit kept as an expected failure, and 1 is an accepted risk.
- **17 findings.** 16 were fixed and each has a regression test; 1 was accepted with a reason (F-17).
- **After the fixes.** AI: 333 tests pass in the full suite, of which 198 are new, with 1 expected failure. Backend: all 51 new tests and subtests pass, and the whole suite passed with the database-backed tests and the race detector.
- **The authentication, authorisation, rate limiting, CORS and error handling held up.** The weak spots were in the AI service: three protections that had been written (masking of personal identifiers, the prompt-injection block and the limit on simultaneous model calls) were never connected to the live endpoint, and the input and output filters had gaps.

The first run of the AI suite had 86 failures. 37 were mistakes in the tests themselves, which we corrected. A recount of the prompt-tag test against the old code showed that 5 of 9 breakout variants really did get through, so that finding stands. The other 49 failures were real defects.

4.3 Findings

Severity is our judgement of impact for this product, not a CVSS score.

Table 6: Findings from the adversarial tests

ID	Severity	Service	Finding	Fix	Regression test
F-01	**High**	AI	National id, e-mail and phone number masking, and the prompt-injection block, existed only in a module that no route used. The live chat sent identifiers straight to the model provider	Moved into the domain guardrail, wired into the live service, and extended to spaced and dashed forms; also applied to the stated goal	AI-17, AI-23, AI-24

continued on the next page

ID	Severity	Service	Finding	Fix	Regression test
F-02	Medium	AI	The limit on simultaneous model calls (`RAG_MAX_CONCURRENT`) was in the same unused module, so a burst of requests was unbounded in cost and load	A per-event-loop semaphore in the live service	AI-36
F-03	Low	AI	In production `/docs` was hidden but `/openapi.json` and `/redoc` were open, handing out the full API surface	All three disabled in production	AI-03
F-04	Medium	AI	No request body size limit	256 KB limit, enforced for declared and streamed bodies	AI-07b
F-05	Medium	AI	Lists and strings in the chat request were unbounded (100,000 holdings made a request take more than 20 s)	Length limits on every list and string	AI-07
F-06	Low	AI	`NaN` and `Infinity` were accepted and printed into the prompt	Refused at validation	AI-08
F-07	Low	AI	A blank (whitespace-only) question was accepted	Refused	AI-05
F-08	Medium	AI	Untrusted text (news, goal, holding names) could close or reopen the `<fakta>` and `<sumber>` data blocks: 5 of 9 variants got through (inner spaces, attributes, zero-width and soft-hyphen characters, fullwidth brackets)	Text is Unicode-normalised first, then any tag-like token, with attributes or spaces, is removed	AI-22
F-09	Medium	AI	An answer that repeated the prompt structure or contained a key or token was returned to the user	Such answers are rejected like any other rule violation (retry, then fallback)	AI-25, AI-26
F-10	Medium	AI	Model output could carry markdown images (a data exfiltration channel), raw HTML and `javascript:` links	Images, HTML and script links are removed from every answer, including source excerpts	AI-29
F-11	Low	AI	Empty answers and answers of any size were returned	Empty is a rule violation; answers are cut at 8,000 characters	AI-30, AI-31
F-12	Medium	AI	The "no buy/sell advice" filter caught 2 of 10 realistic phrasings, and nothing stopped profit guarantees	Broader patterns for commands, "you must", recommendations and guarantees, checked on normalised text; education sentences still pass	AI-33, AI-34
F-13	Low	AI and backend	An inbound `X-Request-Id` of up to 64 arbitrary characters was echoed in the response and written to the logs (log and header injection)	Only plain tokens (letters, digits, `_`, `-`) are accepted; anything else gets a fresh id	AI-16, BE-22
F-14	Low	AI	A profile flag code was free text that went into the prompt; answers and flags were unbounded	Flag codes must be lowercase snake case; at most 50 flags and 200 answers	AI-42, AI-43
F-15	Low	AI	HTML in the model's profile wording was accepted	Markup is rejected, the template is used	AI-44
F-16	Low	Backend	A NUL character in a query value, a path value, or escaped inside a JSON string reached PostgreSQL and produced a 500	Rejected with 400 before the handler; JSON bodies are checked for the escape	BE-02, BE-15, BE-23
F-17	Info	Backend	Registering an address that already has an account answers 409, while login, resend and forgot-password do not reveal accounts	**Accepted**: the message tells a person who forgot they registered to sign in. Registration is limited to 30 per minute per client address	BE-18b (fails if the behaviour changes, so the decision is revisited)

4.4 Where the defects came from

F-01 and F-02 deserve a note. A teammate had written the masking of national id, email and phone numbers, the prompt-injection block and the concurrency limit in a service module, but the router calls a different, older service. The unit tests for those functions passed, so everything looked covered. Only a test that drives the real endpoint and looks at what the model receives could see that none of it ran. That is the main reason these tests go through the HTTP layer.

4.5 How to reproduce

```
# AI service
cd ai && uv sync --extra rag --group dev
uv run pytest tests/test_security_edge.py -q
```

```
# Backend: needs a throwaway PostgreSQL with the migrations applied
cd backend
SEC_TEST_DATABASE_URL=postgres://localhost/finpass_test?sslmode=disable \
  go test -p 1 -count=1 -run TestBE ./internal/app/
```

Run the backend suite with `-p 1` against a database: several packages write to the same tables and would race each other otherwise. The backend tests create and delete users named `sec-*@edge.test`.

4.6 Catalogue of edge cases

Held means the code was already safe. **Found** means a defect, fixed (finding in the last column). **Accepted** is a documented decision. **Limit** is a known limit kept as an expected failure.

Table 7: Edge cases against the AI service

ID	Edge case	Expected	Before	Finding / control
AI-01	8 bad internal keys (empty, blank, wrong, 1 char longer, 1 char shorter, 50 KB, non-ASCII, SQL-like) on all 3 protected routes	401, no 500	Held	Constant-time compare against every accepted key
AI-02	Key sent in the query string or the body instead of the header	401	Held	Only the header is read
AI-03	Production: is the API schema hidden?	`/docs`, `/redoc`, `/openapi.json` all off	Found	F-03
AI-04	`/health` discloses nothing	body has only the envelope	Held	
AI-05	14 invalid chat bodies (empty, blank, 1001 chars, wrong types, bad language, `system` role in history, 13 turns, 4001-char turn, run id with `../`, space, newline, 129 chars)	422	13 held, blank message **found**	F-07
AI-06	9 odd but valid messages (NUL bytes, right-to-left override, zero-width, 1000 emoji, SQL, template and log4shell strings, HTML, control characters)	200, no crash	Held	
AI-07	Payload bombs: 100,000 holdings, 100,000 held kinds, 5,000 recommended assets, 100,000 priorities	rejected fast	Found (timed out at 20 s)	F-05
AI-07b	A 3 MB body, declared by length and streamed in chunks	rejected before parsing	Found (no limit existed, by code review)	F-04
AI-08	`NaN`, `Infinity`, `-Infinity` in a numeric field	422	Found	F-06
AI-09	A 400-digit integer	no crash	Held	
AI-10	JSON nested 100,000 levels deep	4xx	Held	
AI-11	Form body, truncated JSON, invalid UTF-8, empty body, wrong content type	400 or 422	Held	
AI-12	GET/PUT/DELETE/PATCH on the chat route	405	Held	
AI-13	Path traversal and odd paths (`../`, `%2e%2e%2f`, `%00`, `/.env`, `//`)	404, no 500	Held	
AI-14	Turn-trail query abuse (`since=-1`, `abc`, 20-digit number, `1e9`, empty)	no 500	Held	
AI-15	Does run A's trail leak into run B?	no	Held	
AI-16	Inbound request id with 10,000 chars, spaces, quotes and HTML, non-ASCII	echoed id is a plain token of at most 64 chars	Found	F-13
AI-17	7 prompt-injection phrases (English, Indonesian, zero-width space inside the keyword, fullwidth letter)	blocked before the model	Found (7 of 7 reached the model)	F-01
AI-18	A normal regulation question that contains the word "instruksi"	not blocked	Held	No false positive
AI-19	Leetspeak injection (`1gn0re`)	blocked	**Limit**	Expected failure; see Section 4.7
AI-20	The message reaches the model only as the last user turn; a fake `SYSTEM:` line in history stays an assistant turn	yes	Held	
AI-21	Only the last 6 history turns are forwarded	yes	Held	
AI-22	9 ways to close or reopen the `<fakta>` and `<sumber>` data blocks, in 4 fields (news title, news summary, goal, holding name)	no tag survives	Found (5 of 9 variants survived: inner spaces, attributes, zero-width, fullwidth, soft hyphen)	F-08
AI-23	8 personal identifiers in the question and history (national id plain, spaced, dashed; email; phone local, international, spaced, dashed)	never sent to the model or the retriever	Found (8 of 8 sent)	F-01
AI-24	A phone number inside the stated goal	masked	Found	F-01
AI-25	5 answers that leak prompt structure or secrets (`<fakta>` tag, `KUTIPAN SUMBER:`, `FAKTA USER:`, an `sk-or-` key, a bearer token)	never returned	Found (5 of 5 returned)	F-09

continued on the next page

ID	Edge case	Expected	Before	Finding / control
AI-26	A leaky first answer followed by a clean retry	clean one accepted	Found	F-09
AI-27	The API key in logs, errors and `repr` after a failed call	absent	Held	
AI-28	An internal exception with a password in its message	500 with a generic message	Held	
AI-29	Model output with a markdown image to an attacker URL, `<script>`, `<img onerror>`, a `javascript:` link	stripped	Found (4 of 4 passed through)	F-10
AI-30	A 2 MB model answer	cut	Found	F-11
AI-31	An empty or blank model answer	falls back	Found	F-11
AI-32	Forged citation markers (`[100]`, `[1][2][3]` with 2 sources, `[4]`)	never become citations	Held	
AI-33	10 ways to tell the user to buy or sell, or to promise a profit (imperative, "must", "recommend", "guaranteed", "risk-free", zero-width space inside the verb)	refused	Found (8 of 10 passed)	F-12
AI-34	5 plain education sentences ("sebelum membeli", "jual beli saham", ...)	not refused	Held	No false positive
AI-35	A number the sources do not contain	detected	Held	The function works; see Section 4.7 on enforcement
AI-36	40 simultaneous chats	at most `RAG_MAX_CONCURRENT` model calls at once	Found	F-02
AI-37	A model that always breaks the rules	at most 2 model calls per question	Held	Bounded cost
AI-38	Model timeouts	fallback excerpts, no error	Held	
AI-39	7 malformed provider responses (HTML page, empty choices, null content, list content, truncated JSON, 429, 500)	`LLMError` only	Held	
AI-40	5 pathological answers of 50 KB (unclosed images, brackets, digit dots, repeated phrases)	each filter under 1 s	Held	No ReDoS in the answer filters
AI-41	Masking is fast on the longest allowed inputs	under 1 s	n/a (no masking ran in the live path)	F-01; no ReDoS in the new patterns
AI-42	Free text in the profile request (name, answers, flag code) reaches the model	no	Found (a flag code went into the prompt as free text)	F-14
AI-43	8 invalid or oversized profile requests (extra field, bad enums, negative income, 5,000 answers, 5,000 flags)	422	6 held, 2 found	F-14
AI-44	5 hostile model replies for the profile text (script tag, 5,000 chars, missing key, wrong type, invented number)	template fallback	4 held, script tag **found**	F-15
AI-45	Non-JSON model reply for the profile	template fallback	Held	

Table 8: Edge cases against the backend

ID	Edge case	Expected	Before	Finding / control
BE-01	4 SQL payloads in login fields (`' OR '1'='1`, `DROP TABLE`, `UNION SELECT`)	401 or 422, table intact	Held	Parameterised queries (sqlc)
BE-02	SQL and control bytes in the search parameters (7 payloads incl. `%00`, 5,000 chars)	no 500	Found (`%00` gave 500)	F-16
BE-03	A name like `Robert'); DROP TABLE users;--`	stored as plain text	Held	
BE-04	Forged tokens: `alg:none` (library, hand built, no signature segment), tampered payload, garbage, empty segments, 100 KB	401	Held	Only HS256 is accepted
BE-05	A token signed with another secret, or with HS512	401	Held	
BE-06	Expired token, token with no expiry	401	Held	Expiry is required
BE-07	Wrong issuer	401	Held	

continued on the next page

ID	Edge case	Expected	Before	Finding / control
BE-08	Empty subject, unknown user, SQL in the subject	401	Held	
BE-09	9 `Authorization` variants, token in the query string, token in a cookie	401	Held	
BE-10	User B patches, deletes and lists user A's holding; tries to reassign it with `userId`	404, nothing changes	Held	Ownership is part of every query
BE-11	9 failed logins with a different `X-Forwarded-For` each time	429 after 5; the header changes nothing	Held	Proxy headers ignored unless a proxy count is configured
BE-16	Passwords: 7, 72, 73 chars, 144-byte multi-byte, emoji, spaces, NUL, 10 KB, SQL, combining marks	never a 500	Held	Rune count and bcrypt byte limit both handled
BE-17	E-mail case and spacing, 250-char and non-ASCII addresses	one account per address	Held	
BE-18	Do login, resend and forgot-password reveal whether an address has an account?	identical responses	Held	
BE-18b	Does registering an existing address reveal it?	identical	Accepted	F-17
BE-19	8 wrong verification codes	locked after 5, account not verified	Held	
BE-12	A 2 MB body	4xx, not read in full	Held	1 MB cap
BE-13	Mass assignment (`email_verified`, `id`, `role` in the body)	refused, or the flag ignored	Held	Unknown fields are rejected
BE-14	JSON nested 200,000 levels	4xx	Held	
BE-15	12 malformed bodies (empty, array, null, NaN, two objects, wrong types, form body, escaped NUL ...)	4xx	11 held, escaped NUL (`\{}u0000`) **found**	F-16
BE-20	7 hostile origins (`null`, look-alike hosts, other ports, https) on GET and preflight	no CORS headers	Held	The configured origin is allowed; never `*`
BE-21	Security headers on 404, 405, 401, 422, health, preflight and bad requests	all present	Held	`Cache-Control: no-store` on personal data
BE-22	The same hostile request ids as AI-16	plain token	Found	F-13
BE-23	8 odd paths and hostile ids (traversal, double encoding, 3,000 segments, `%00`)	no 500	Found (`%00` gave 500)	F-16
BE-24	Method override header, TRACE, CONNECT, PROPFIND, PATCH, DELETE on a read route	no 500, no state change	Held	
BE-25	Old access token after the account was deleted	401	Held	
BE-26	Forged refresh cookies; cookie flags	401; HttpOnly and SameSite set; refresh token not in the body	Held	
BE-27	11 deliberately broken requests across profile, dashboard, news, discover, advisory	no database error, stack trace, path or secret in any response	Held	Checked on **every** response of every backend test
BE-28	Sign-in before the e-mail is verified	refused	Held	
BE-29	Data export called 5 times	limited	Held	3 per hour

4.7 What this testing does not show

- **Jailbreaks.** No filter stops every attempt to steer a language model. The design limits the damage instead: rules, not the model, decide; untrusted text is fenced as data; answers are checked before they are shown; personal identifiers are masked; and one question costs at most two model calls. Case AI-19 (leetspeak such as `1gn0re`) is kept as an expected failure so that this limit stays visible.
- **Invented numbers.** The function that detects a figure absent from the sources works (AI-35), but the setting `RAG_ENFORCE_NUMBERS` is not read by the live chat, and the existing tests state that model-computed numbers are allowed. The profile explanation does enforce its number check.
- **Request-rate limiting in the AI service.** There is none, by design: only the backend can call it, using a shared secret, and the backend limits chats per user. The AI service limits concurrency (F-02) and body size (F-04), not request rate.
- **The browser.** How the frontend renders model text, and whether it could execute anything, was not tested. The AI service now strips images, HTML and script links from answers (F-10), which removes the main risk at the source.
- **Timing attacks** on the key comparison were reviewed in the code (constant time) but not measured. The **WhatsApp webhook** signature check is covered by the existing tests of that package and is not repeated here. **Load and capacity** were not tested: concurrency was checked for correctness, not throughput. **Real providers** (OpenRouter, Cloudflare R2, SMTP, the WhatsApp gateway) were faked.

4.8 Recommendations

1. Keep these tests in CI. The AI ones need no network; the backend ones need a database and `-p 1`.
2. Decide whether `RAG_ENFORCE_NUMBERS` should be wired into the live chat. It would reduce invented figures at the cost of more fallback answers.
3. Revisit F-17 if account discovery becomes a concern; its test fails when the behaviour changes.
4. Add a browser-level check that model text is rendered as plain text.
5. When a protection is added, add a test that drives the real route. A unit test of an unused function gives false comfort (see F-01).

5 Remediation roadmap

Six risks are fixed (R-01, R-05, R-14, R-19, R-20, R-21), and eleven are reduced with a stated gap. This section lists what remains. It is a proposal: the team has not scheduled or committed to any item, and the effort sizes are our rough guess (S is under a day, M is a few days, L is a week or more). Some items need access we do not have from the code, such as the hosting dashboard, and we say so.

Table 9: Remaining work, in the order we would take it

Risks	Change	Effort
R-10, R-12	Read back the deployed environment and compare it with the start-up checks. Confirm the AI service has no public address. Turn the localhost CORS warning into a failure. Needs the hosting dashboard.	S
R-06	Confirm how many proxies sit in front of the API and set `TRUSTED_PROXY_HOPS`. Needs the hosting dashboard.	S
R-09	Split env files per service and rotate every secret that sat in the wrong file. Then remove the previous-key setting.	M
R-15, R-03	Set a spend cap at the model provider and pin providers in production. Needs the provider dashboard.	S
R-13	Fail, not warn, when a manifest has no hashes. Restrict write access to the corpus bucket.	S
R-02, R-04	Grow the recommendation phrase list from reviewed answers, add an injection test set, and drop instruction-like lines from news before indexing.	M
R-08	Seal holdings, and decide about email and name.	M
R-06, R-07, R-15	Move the in-memory counters to a shared store when the API runs on more than one instance.	M
R-17	Review the report-only CSP violations in a browser, then enforce, then move to nonces.	M

Risks	Change	Effort
R-18	Add TOTP for password accounts.	L
R-11	Replace the unofficial WhatsApp gateway with the official Business API and review the vendored code.	L
R-22	Re-run the dependency scan when fixed ChromaDB and NLTK versions appear, and upgrade.	S

Rotating the internal key

The AI service accepts a second key, `INTERNAL_API_KEY_PREVIOUS`, so the shared key can change without downtime. Set the new key as `INTERNAL_API_KEY` on the AI service and put the old one in `INTERNAL_API_KEY_PREVIOUS`. Then change the key the API sends, and finally remove the previous key from the AI service.

Row-level security in PostgreSQL would back up the `user_id` predicates in the application. We did not list it as a risk because we found no query missing the predicate, but we would add it before the data set grows.

6 Limitations of this self-assessment

We read the code, ran the existing tests, and wrote 76 hostile-input test cases against the backend and the AI service (Section 4). Those cases ran against the code, not against the deployed system. A penetration test, or even a single outside reader, would find things we missed, and we have not had either.

We assessed the repository, not the deployment. We did not look at Railway settings, network rules, the database's public exposure, the R2 bucket permissions, the SMTP account, the GoWA host or the model provider's account. Every risk that depends on those is marked Not verified. The local `.env.production` files are not a record of what runs in production, and we read only their variable names and a few non-secret settings, never the secret values.

We did not run `gosec` or `trivy`. We ran the full Go test suite (against a local PostgreSQL instance, so the database-backed tests ran), the Python tests, `govulncheck`, `pip-audit`, `bandit` and `npm audit` on October 20, 2026, after the fixes in Section 3. A clean result on that day says little about next week. The authentication transport package has no unit tests of its own; the adversarial tests exercise it through the router. We could not check the browser behavior of the frontend, which is why the CSP is still report-only.

We did not review the vendored GoWA project, third-party Go, Python or npm packages, the frontend components for cross-site scripting beyond a search for unsafe HTML sinks, or the correctness of the financial rules. This report covers security. It says nothing about whether the profile scoring or product matching is financially sound.

The ratings in Section 3 are our judgment. Another reviewer could rank them differently, and we would expect them to move R-02 and R-03 up if the product reaches users in a regulated setting. Whether Finpass counts as financial education or licensed advice, and which regulation applies, is a legal question we have not answered.

The code reading and the first draft of this report were done with an AI coding assistant (Claude Code) working through the repository. Every statement about the code carries a file reference so a person can confirm it, and several items are listed in `open-questions.md` because only the team can answer them.